Decimal Literal:

By default, our numeric literals are decimal literal. Here base is 10 so, It accepts 10 digits i.e. from 0-9.

Example:
```
int x = 20;
int y = 123;
int z = 234;
```

Octal Literal:

If any Integral literal starts with 0 (Zero) then it will become octal literal. Here base is 8 so it will accept 8 digits i.e. 0 to 7.

Example:
```
int a = 018; //Invalid because it contains digit 8 which is out of range
int b = 0777; //Valid
int c = 0123; //Valid
```

Hexadecimal Literal:

If any integral literal starts with 0X or 0x (Zero capital X Or 0 small x) then it is hexadecimal literal. Here base is 16 so it will accept 16 digits i.e. 0 to 9 and A to F OR [a to f]

Example:

```
int x = 0X12; //Valid
int y = 0xadd; //Valid
int z = 0XFACE; //valid
int a = 0Xage; //Invalid because character 'g' out of range
```

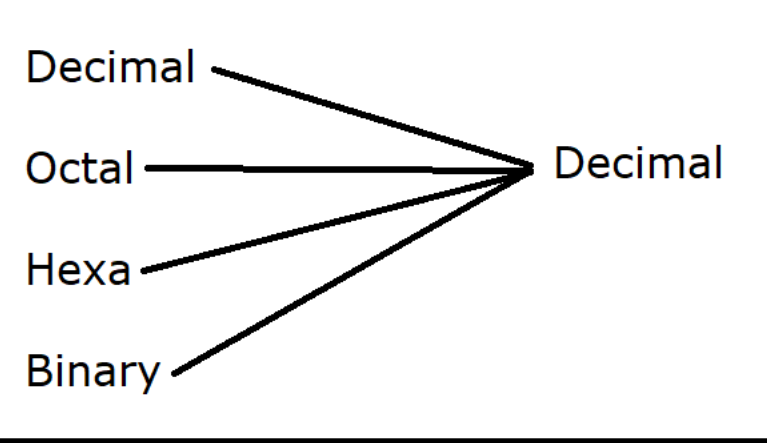
Binary Literal:

If an Integral literal starts with 0B (Zero capital B) or 0b (Zero small b) then it will become Binary literal. Binary literal is available from JDK 1.7v.
Here base is 2 so it will accept 2 digits i.e. 0 and 1 only.

Example:

```
int x = 0B101; //valid
int y = 0b111; //Valid
int z = 0B112; //Invalid [2 is out of range]
```

As a developer, we can represent an integral in four different forms (Decimal, Octal, hexa and binary) but to get the result OR output, JVM will convert into decimal format only because decimal is the default number system.



2 1 0

523

500 + 20 + 3

$5X10^{2} + 2X10^{1} + 3X10^{0}$

```
//Octal Literal
void main()
{
    int x = 015;
    IO.println(x);
}
```

015 -> Here '0' represents that it is an octal literal.

1 0

$(1\ 5)_{8} = (?)_{10}$

$1X8^{1} + 5X8^{0}$

8 + 5 X 1

8 + 5 = 13

```
//Hexadecimal Literal
void main()
{
    int x = 0Xadd;
    IO.println(x);
}
```

0Xadd : Here 0X describes that It is hexadecimal literal

2 1 0

$(a\ d\ d)_{16} = (\ ?\ )_{10}$

$aX16^{2} + dX16^{1} + dX16^{0}$

10 X 256 + 13 X 16 + 13 X 1

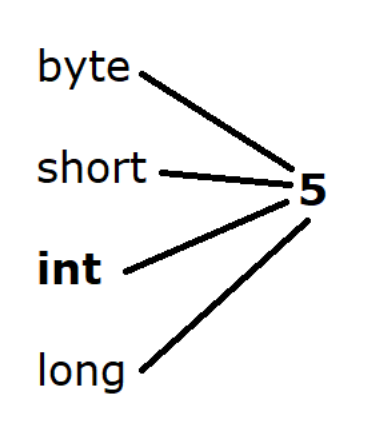
2560 + 208 + 13 = 2781

a = 10
b = 11
c = 12
d = 13
e = 14
f = 15

```
//Binary Literal
void main()
{
    int x = 0B111;
    IO.println(x);

    int y = 0B101;
    IO.println(y);
}
```

By default every integral literal is of type **int** only.



by default 5 is of int type.

Memory representation of Integral literal data type :

```
byte b = 5;
00000101 //8 bits


short s = 5;
00000000  00000101 //16 bits


int x = 5;
00000000 00000000 00000000 00000101  //32 bits


long y = 5;
00000000 00000000 00000000 00000000   00000000 00000000 00000000 00000101 //64 bits
```

* by default javac will represent any integeral literal in 32 bits representation.

byte range :   -128 to 127
short range :   -32768 to 32767

By default, every integral literal is of type **int** only. byte and short are below than int so we can assign integral literal (Which is by default int type) to byte and short but the values must be within the range. [for byte -128 to 127 and for short -32768 to 32767]

Actually, whenever we are assigning integral literal to byte and short data type then compiler internally converts into corresponding type.

byte b = (byte) 12; [Compiler is converting int to byte]
short s = (short) 12; [Compiler is converting int to short]

In order to represent long value explicitly we should use either L OR l (Capital L OR Small l) as a suffix to integral literal.

According to IT industry, we should use **L** because l (small l) looks like digit 1.

Case 1 :
----
byte b = 90; [Here Compiler will convert int (90) to byte but value must be within the range only -128 to 127]

Case 2 :
----
short s = 125; [Here Compiler will convert int (125) to short but value must be within the range only -32768 to 32767]

Case 3 :
----
int x = 12; [32 bits to 32 bits]

Case 4 :
----
long l = 90; [32 bits to 64 bits]

Case 5 :
----
long mob = 9812345678; //error [Interger number too large]

Case 6 :
----
long mob = 9812345678L; //32 bits converted into 64 bits to hold the mobile number

Case 7 :

```
void main()
{
  accept(1);
}

 void accept(byte b)
 {
  IO.println(b);
 }
```